brain/show_carl.py — Full Line-by-Line Walkthrough

Length: 44 lines. **Role:** a tiny standalone utility script — not part of the cognitive control loop at all — that loads CARL's physical body into MuJoCo's interactive 3D viewer, purely so a human can look at the robot and its surrounding objects (food, obstacle, predator) without running the full sensing/brain/motor pipeline. Think of it as "open the model in a 3D viewer and let me spin the camera around it," useful for checking the body's geometry and object placement while iterating on the world XML.

Lines 1–6: Imports and path setup

```python
import mujoco
import mujoco.viewer
import numpy as np
import os as _os
_FILE_DIR = _os.path.dirname(_os.path.abspath(__file__))
BASE_DIR = _os.path.dirname(_FILE_DIR)
```

Same portable-path pattern seen in `carl_simulation.py`: resolve this script's own folder, then step up one level to the repository root (`BASE_DIR`), so the script works regardless of which machine or working directory it's run from. `os` is imported under the alias `_os` — the underscore prefix is a common convention meaning "this name is a private implementation detail, don't treat it as part of this module's public interface," but here it also creates a real functional issue described below. `numpy` is imported but never actually used anywhere in the file — dead import, harmless but unnecessary.

Line 10: The bug — `os` vs `_os`

```python
xml_path = os.path.join(BASE_DIR, "world", "vessel_kinetic.xml")
```

This line calls `os.path.join`, but the import at the top only defined `_os`, not `os`. Since nothing else in the file (or, typically, in Python's implicit builtins) defines a bare name `os`, calling `show_carl()` will raise a `NameError: name 'os' is not defined` at exactly this line, before the model even loads. This is a genuine, currently-broken bug in the script as written — every other reference to the path helper elsewhere in the file correctly uses `_os` (see line 6), so this looks like a simple copy-paste slip from a version of the file that imported `os` directly rather than aliasing it. The fix is a one-character change: `_os.path.join(...)`.

Lines 9, 11–13: Function definition and model load

```python
def show_carl():
    xml_path = os.path.join(BASE_DIR, "world", "vessel_kinetic.xml")
    print(f"Loading model from {xml_path}...")
    model = mujoco.MjModel.from_xml_path(xml_path)
    data = mujoco.MjData(model)
```

Assuming the `os`/`_os` bug were fixed, this loads the exact same physics world (`vessel_kinetic.xml`) that `carl_simulation.py`'s `SimulationHAL` loads — confirming this really is meant as a lightweight companion viewer for the same robot body, not a different one. `MjModel` is the static description; `MjData` is the mutable runtime state, same distinction covered in the main simulation walkthrough.

Lines 15–17: Center the robot

```python
q_addr = int(model.joint("spatial_identity").qposadr[0])
data.qpos[q_addr : q_addr+3] = [0.0, 0.0, 0.05]
```

Looks up the free joint that gives the robot's root body its world position (same joint name, `spatial_identity`, used throughout `carl_simulation.py`), and places the robot at the origin, 5cm above the ground — a fixed, known starting pose so whoever runs this viewer always sees CARL in a predictable spot rather than wherever the XML file's default pose happens to be.

Lines 19–24: Move the interactive objects in close

```python
model.geom_pos[model.geom("food").id] = [0.5, 0.5, 0.12]
model.geom_pos[model.geom("obstacle").id] = [-0.8, 0.6, 0.45]
model.geom_pos[model.geom("obstacle_glow").id] = [-0.8, 0.6, 0.904]
model.geom_pos[model.geom("predator").id] = [0.0, -0.6, 0.15]
model.geom_pos[model.geom("predator_glow").id] = [0.0, -0.6, 0.46]
```

By default these objects might be scattered far across the arena (as they would be during an actual simulation run, where `respawn_food()` places food randomly). For a "just look at the robot" viewer, that's inconvenient — so this script hard-codes all of them to sit within about a meter of the robot's fixed starting position, so a human opening the viewer immediately sees CARL next to food, an obstacle, and a predator without having

to search the arena. The `_glow` geoms (`obstacle_glow`, `predator_glow`) are a detail not seen in the other walkthroughs: these are presumably separate, larger/translucent geoms layered visually on top of the solid obstacle/predator bodies — a glow or halo effect for visibility — and each is manually kept at the same (x, y) as its solid counterpart but at a different height (`0.904` vs `0.45` for the obstacle, `0.46` vs `0.15` for the predator), meaning the glow effect sits noticeably higher than the solid object itself, likely so it reads visually as a halo hovering around/above the object rather than being hidden inside it.

Line 27: Recompute derived state

```python
mujoco.mj_forward(model, data)
```

After manually overwriting several positions directly in `qpos`/`geom_pos`, MuJoCo's derived quantities (world transforms, contact detection, etc.) don't automatically update — `mj_forward` is what actually recomputes the physics engine's internal state from the raw position data, the same call used in `SimulationHAL.reset_robot()` and `respawn_food()` for the same reason: make the manual edits "take effect" before anything (here, the viewer) reads the model.

Lines 29–38: Cosmetic console banner

```python
print("\n============================================================")
print("Launching MuJoCo 3D Interactive Viewer!")
print("------------------------------------------------------------")
print("Controls:")
print(" - Left Click + Drag: Rotate Camera")
print(" - Right Click + Drag: Pan Camera")
print(" - Scroll Wheel: Zoom In/Out")
print(" - Double Click on Object: Focus Camera on Object")
print(" - Spacebar: Pause/Resume Physics")
print("============================================================\n")
```

Pure user-facing documentation printed to the terminal — a quick-reference for MuJoCo's built-in viewer mouse/keyboard controls (these are the viewer's own generic default bindings, not anything custom-coded in this repository). No effect on simulation or rendering behavior.

Line 40: Launch the blocking viewer

```python
mujoco.viewer.launch(model, data)
```

This is the important functional difference from `carl_simulation.py`'s `mujoco.viewer.launch_passive(...)` call. `launch_passive` hands control back to the calling Python code immediately, letting the caller drive the physics loop itself (exactly what `run_simulation()`'s `while` loop does, calling `mj_step` manually every tick). `launch()` (no `_passive`) is *blocking* — it takes over the whole program, runs its own internal physics loop, and only returns once the viewer window is closed. That's the right choice here: this script has no cognitive loop of its own to run, it's purely "load this model and let MuJoCo's own default stepping/viewer handle everything until the user closes the window."

Lines 42–43: Script entry point

```python
if __name__ == '__main__':
    show_carl()
```

Standard guard so `show_carl()` only runs when this file is executed directly (`python show_carl.py`), not if something elsewhere imports it.

Summary

A small, single-purpose visualization helper: load the body, park it and its nearby objects in a fixed, easy-to-see arrangement, and hand the whole thing over to MuJoCo's own interactive viewer loop — no sensing, no brain, no motors, just "let me look at it." As written, it currently has one real bug (`os.path.join` should be `_os.path.join` on line 10) that would need a one-line fix before the script would actually run.